Edge-Assisted Post-Quantum Secure Pipeline for Remote Forensic Data Collection with Hyperledger Fabric

Aekkarin Yimyaem, Nattapol Phanmadee, Pattarapong Tiemtan, Somchart Fugkaew, Uchupa Pao-in

Sirindhorn International Institute of Technology, Thammasat University, Thailand

6522772498@g.siit.tu.ac.th, 6522781994@g.siit.tu.ac.th, pat.tiemtan@pm.me, somchart@siit.tu.ac.th, 6522770666@g.siit.tu.ac.th

***Abstract*- The collection and analysis of remote forensic data are critical for ensuring cybersecurity, investigating incidents, and maintaining system integrity. However, gathering sensitive data from distributed edge devices and processing it in cloud environments raises significant concerns regarding privacy, integrity, and post-quantum security. Traditional encryption methods may not withstand quantum attacks, while naive cloud storage solutions often compromise confidentiality. To address these challenges, we propose an Edge-Assisted Post-Quantum Secure Pipeline for Remote Forensic Data Collection that integrates lightweight post-quantum symmetric encryption with Hyperledger Fabric for immutable, auditable storage. Our approach encrypts each piece of evidence with a unique post-quantum symmetric key, which is optionally wrapped with AES and protected using Ciphertext-Policy Attribute-Based Encryption (CP-ABE) to enforce fine-grained access control. Collected data and cryptographic metadata are recorded on a Hyperledger Fabric ledger, ensuring tamper-proof provenance and traceable audit trails. To improve system efficiency, we incorporate indexing mechanisms to support scalable data retrieval and real-time forensic analytics. This integrated framework provides a secure, auditable, and high-performance solution for managing distributed forensic data in untrusted cloud environments, resilient against both classical and quantum adversaries.**

Keywords- Remote Forensic Data, Edge Computing, Post-Quantum Cryptography, Ciphertext-Policy Attribute-Based Encryption, Hyperledger Fabric, Immutable Ledger, Symmetric Encryption, Key Management

I. Introduction

The collection and analysis of forensic data from remote or distributed systems are critical components of modern cybersecurity and incident response frameworks. Digital evidence plays an increasingly crucial role in judicial proceedings, but its volatility and susceptibility to tampering necessitate robust mechanisms to ensure integrity and authenticity [8]. Failure to maintain a verifiable chain of custody (CoC) can render evidence inadmissible in court, highlighting the need for robust digital evidence management systems [9]. Timely acquisition of system logs, network events, and other digital evidence enables organizations to detect malicious activity, investigate breaches, and maintain overall system integrity. However, gathering sensitive data from geographically distributed edge devices and processing it in cloud environments presents multiple challenges, including data privacy, secure transmission, and protection against emerging quantum threats.

Traditional encryption schemes, while effective against classical attacks, are vulnerable to quantum algorithms, necessitating the adoption of post-quantum cryptographic (PQC) techniques. In this work, each piece of evidence is encrypted with a unique post-quantum symmetric key, which is optionally wrapped with AES and further protected using Ciphertext-Policy Attribute-Based Encryption (CP-ABE) to enforce fine-grained access control prior to cloud storage.

The integration of a distributed ledger, such as Hyperledger Fabric, provides tamper-proof storage and auditable tracking of collected forensic events, ensuring data integrity and enhancing trust in multi-party systems. Blockchain-based smart contracts have been shown to automate multi-party processes efficiently, supporting decentralized execution and auditable transaction management across organizational boundaries [1]. While highly suitable for enterprise and forensic applications, deploying a Fabric network involves configuring multiple components, which can be challenging; automated deployment frameworks such as AHFD help reduce this complexity and streamline network setup [10].

This work proposes an edge-assisted, post-quantum secure pipeline that combines these technologies to enable secure, efficient, and scalable collection, transmission, and storage of forensic data in untrusted cloud environments. By leveraging PQC, blockchain, and advanced data processing techniques, the proposed framework addresses the dual challenges of security and performance inherent in distributed forensic data collection.

II. Related Work

This section discusses prior work on secure forensic data collection, including post-quantum cryptography, blockchain-based logging and access control, and efficient indexing techniques for distributed datasets, highlighting their relevance to our proposed pipeline.

In [2], Alkhansaa Abuhashim et al. explored various smart contract designs to facilitate indexing and querying of blockchain data. Their work evaluated Catalog and Sparse smart contracts for efficient retrieval of transactions, highlighting trade-offs in query complexity and ledger scanning. While these designs improve blockchain data accessibility, they do not address the challenges of encrypting sensitive information or enforcing fine-grained access control in distributed multi-party systems.

In [3], Lee and Lee proposed a blockchain-based RBAC mechanism that integrates individual authentication with role-based access control while preserving user anonymity. Their approach leverages smart contracts to enforce permissions and supports efficient verification on a distributed ledger. While effective for access control and privacy, this method does not address post-quantum security or the encryption of sensitive forensic data in edge-assisted systems.

In [4], Shahid et al. proposed a novel one-time signature (OTS) scheme for hash-based post-quantum digital signatures, achieving significant reductions in key and signature sizes compared to existing schemes such as

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Winternitz OTS and WOTS+. While their approach enhances efficiency for PQ digital signatures, it does not address the challenges of fine-grained access control or encrypted forensic data collection in distributed edge-assisted systems.

In [5], Zou and Zeng introduced the expander signature, a digital signature primitive that allows a signer to generate multiple signatures in advance and release verification keys as needed. Their scheme enables efficient signature management even on resource-constrained devices and can be applied to blockchain systems for secure transaction verification. While it improves digital signature efficiency and integrity, it does not directly address post-quantum secure encryption or fine-grained access control for distributed forensic data.

In [6], Seno et al. proposed CDBS, a post-quantum blind signature scheme for consumer IoT applications based on the CRYSTALS-Dilithium algorithm. Their approach provides quantum-resistant authentication while preserving privacy through blind signing, supporting secure transactions and interactions in distributed IoT systems. Although effective for post-quantum authentication, this scheme does not directly address encrypted data storage, fine-grained access control, or edge-assisted forensic data collection.

In [11], Bálint demonstrated a permissioned Hyperledger Fabric network for secure storage of sensitive renewable energy management data. The study illustrated how Fabric can enforce access control, prevent unauthorized modifications, and support multi-party data sharing in a distributed system. While focused on energy management, the work highlights practical considerations for deploying Fabric networks to protect sensitive information, which can inform the design of forensic data pipelines.

In[14], Ahmed et al. proposed a blockchain-enabled healthcare data-sharing framework that integrates CP-ABE, IPFS decentralized storage, and Ethereum smart contracts to enhance data integrity and fine-grained access control in Industry 5.0 environments. Their system introduces human-centric authentication, decentralized pseudo-identities, and energy-efficient Proof-of-Stake consensus, achieving notable improvements such as 32 ms key generation, 27% reduced computational overhead, and 99.5% resistance to MITM attacks. While their approach strengthens secure data exchange for medical IoT systems, it does not address post-quantum secure forensic evidence protection, chain-of-custody guarantees, or hybrid encryption pipelines required for tamper-resistant remote forensic data collection. Additionally, the use of Ethereum and IPFS focuses on healthcare record management rather than supporting privacy-preserving events, evidence life-cycle controls, or cross-stakeholder forensic access policies as required in forensic investigations.

III. Related Knowledge

A. XChaCha20-Poly1305

XChaCha20-Poly1305 is an extended-nonce variant of the ChaCha20-Poly1305 authenticated-encryption cipher. It uses a 256-bit key and a 192-bit nonce to avoid collision risk: the first 128 bits of the nonce are combined with the key via the HChaCha20 hash function to derive a one-time subkey, and the remaining 64 bits (plus a 32-bit block counter) are used with standard ChaCha20-Poly1305 encryption.The practical effect is that each message can use a random 192-bit nonce without fear of accidental reuse; for example, a 192-bit nonce will, under ideal randomness, incur a 50% chance of collision only after on the order of 2^96 messages.Implementations of XChaCha20-Poly1305 (such as in libsodium, WireGuard, or Go's crypto library) exploit this design to avoid expensive misuse-resistant constructions while still preventing nonce-reuse attacks.In secure systems, XChaCha20-Poly1305 provides high-speed, low-latency encryption: it has no large lookup tables (avoiding timing attacks) and its performance is comparable to ChaCha20. From a post-quantum perspective, XChaCha20's 256-bit key remains effectively secure against quantum attackers which can at best mount a Grover-search speedup, so it can continue to protect data integrity and confidentiality in a post-quantum pipeline.

B. Ciphertext-Policy Attribute-Based Encryption (CP-ABE)

Ciphertext-Policy ABE is a public-key encryption paradigm for fine-grained access control. In a CP-ABE scheme, each ciphertext is labeled with an access policy typically expressed as a monotonic Boolean formula or tree over attributes, and each user's private key is issued based on a set of attributes. A user can decrypt a ciphertext if and only if their attribute set satisfies the ciphertext's policy . Thus, the encryptor rather than a central authority directly controls who may decrypt by embedding the policy in the ciphertext. A key security feature is collusion resistance. No coalition of users can combine their keys to decrypt a ciphertext unless at least one user individually already meets the policy requirements. Early CP-ABE constructions achieve this using bilinear pairings and secret sharing, so that decrypting under multiple keys gives no advantage. CP-ABE is well suited to scenarios like encrypted cloud storage or IoT data sharing, where data must remain protected on untrusted servers but only users with appropriate roles or credentials can decrypt. For example, a healthcare record could be encrypted under a policy like "(Doctor AND Cardiology) OR (Patient AND OwnRecord)", and only users whose issued attributes (e.g. "Doctor", "Cardiology") satisfy that policy can access the record . (Current CP-ABE schemes are based on classical groups and would be vulnerable to large quantum computers unless replaced by lattice-based ABE variants, but they remain powerful tools for enforcing privacy-preserving access control today.)

C. Kyber

CRYSTALS-Kyber is a lattice-based Key Encapsulation Mechanism (KEM) that has been selected by NIST for standardization as FIPS 203 (renamed "ML-KEM" for Module-Lattice KEM). Kyber's security relies on the hardness of the Module Learning With Errors (MLWE) problem, a generalization of the LWE problem over module lattices.In operation, Kyber generates a public/secret key pair and allows one party to encapsulate a random symmetric key under the recipient's public key. The underlying algorithm first constructs a public-key encryption (PKE) scheme from MLWE, then applies the

Fujisaki–Okamoto (hashbased) transform to achieve IND-CCA2 security, yielding the KEM. NIST's announcement emphasizes that Kyber (ML-KEM) has comparatively small key and ciphertext sizes and high performance. In practice, Kyber enables two parties to agree on a 256-bit shared secret securely: the sender encapsulates a random 256-bit key and sends the short ciphertext; only the holder of the correct private key can decapsulate and recover the key. Since Kyber is designed to resist known quantum algorithms, it is suitable as a drop-in replacement for classical Diffie–Hellman or RSA-based key exchange in protocols like TLS, VPNs, or hybrid encryption systems. Its selection as the primary post-quantum encryption standard (MLKEM) means it will see wide adoption for securing communications against future quantum attacks.

D. Dilithium

CRYSTALS-Dilithium is a lattice-based digital signature scheme chosen by NIST as the primary post-quantum signature standard.Dilithium signatures are built on the hardness of MLWE and a related Module Short Integer Solution (MSIS) problem in structured lattices. Conceptually it is a Schnorr-style signature: the signer generates a randomized commitment (sampling short lattice vectors) and produces a challenge via a hash (Fiat–Shamir), yielding a short signature vector. NIST notes that ML-DSA (Dilithium) is believed secure even against largescale quantum computers. Compared to other post-quantum signatures, Dilithium offers relatively compact public keys and moderate signature sizes, with fast signing and verification. For example, security proofs assume the adversary can be modeled by MLWE hardness in the random-oracle model, giving confidence in the scheme. In secure systems, Dilithium can replace RSA/ECDSA: it provides non-repudiable authentication of messages or transactions, and its modular-lattice basis means that a quantum adversary would only achieve a subexponential (not polynomial) break, preserving long-term integrity.

E. Hyperledger Fabric

Hyperledger Fabric is an open-source, permissioned blockchain framework designed for enterprise and decentralized applications. Unlike public blockchains, Fabric operates among a known consortium of participants identified via PKI certificates. It features a modular architecture: consensus mechanisms, membership services, cryptographic libraries and even ledger storage can be "plugged in" to meet specific trust models . Fabric does not require a native cryptocurrency, instead it uses lightweight consensus tailored to the application. Fabric's programming model separates transaction endorsement from ordering and commit phases, and it allows smart contracts ("chaincode") to be written in standard languages (Go, Java, etc.). Critically, Fabric provides fine-grained confidentiality: transactions can be confined to channels or private data collections so that only authorized peers see sensitive content. These properties make Fabric attractive for securing distributed workflows and forensic logs. For instance, in a supply-chain or forensics pipeline Fabric could record hashes or attestations of data uploads, ensuring immutability and auditability without exposing raw content to all parties. Its identity and endorsement policies guarantee that only authenticated organizations can append to the ledger . In summary, Hyperledger Fabric combines blockchain-style tamper-evidence and decentralization with enterprise-grade permissioning and privacy controls, making it a key component in secure distributed systems.

F. Proxy re-encryption (PRE)

Proxy Re-Encryption (PRE) is a public-key cryptographic primitive that enables a semi-trusted proxy to convert a ciphertext under one user's key into a ciphertext under another user's key without ever learning the underlying plaintext. In practice, this allows an evidence custodian (delegator) to generate a re-encryption key that lets the proxy re-encrypt stored forensic data so that a designated investigator (delegatee) can decrypt it, all without exposing raw evidence or secret keys.This mechanism naturally supports secure access delegation and revocation: access can be granted by issuing a re-encryption key and later revoked by withholding or replacing it, rather than re-sharing data or private keys. The proxy also cannot obtain the plaintext during re-encryption," and that permissions (i.e. delegation rights) "can be conveniently changed" to revoke access when needed.In digital forensics systems, PRE has been proposed to strengthen the evidence chain-of-custody encrypted case files can be passed from one custodian to another via re-encryption, preserving confidentiality and integrity throughout handoffs. In cloud or multi-tenant forensic databases, this means encrypted evidence can be flexibly shared among investigators and agencies with access strictly controlled by re-encryption keys without any proxy ever seeing the cleartext evidence. Such PRE-based schemes thus provide fine-grained, cryptographically-enforced delegation of encrypted evidence, aligning with secure handoff and access-control requirements in modern distributed forensics environments

E. Merkle Mountain Range

A Merkle Mountain Range (MMR) is an append-only cryptographic data structure designed for environments where data continuously grows, such as blockchains or forensic event logs. Conceptually, an MMR is a forest of perfect Merkle trees ("mountains") arranged from left to right. Each time new data arrives, it is appended as a new leaf, and the structure automatically grows by forming the largest possible perfect tree at each step. This creates a sequence of peaks, each peak being the root of a perfect binary tree, whose combined hashes form the MMR root.

The key advantage of an MMR is that it supports efficient historical proofs without requiring storage of past tree versions. Since each leaf's position in the structure is deterministic, a client can later provide a compact Merkle proof showing that some piece of data existed at a specific index, even if the dataset has grown far beyond that point. Importantly, adding new leaves never invalidates older proofs because the MMR never restructures past nodes. This makes MMRs particularly useful in forensic systems, blockchains, and decentralized protocols where tamper-evident, append-only logging is essential.

IV. Our Proposed Scheme

A. System Overview

Our proposed system integrates blockchain technology, quantum-resistant encryption, and policy-driven access control to secure the entire lifecycle of digital evidence. The architecture ensures confidentiality, integrity, and transparency through five major components: (1) Evidence Collection, (2) Key Generation, (3) Encryption, (4)

Retrieval Indexing, and (5) Policy Enforcement via Smart Contracts. Figure 1 illustrates the proposed system model.

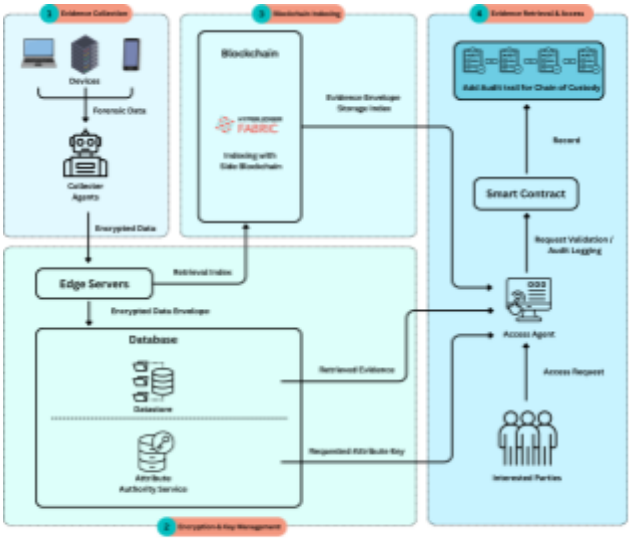


Fig.1 Our System Model

Algorithm 1 Evidence Collection

Procedure EvidenceCollectionAgent()
  config, collector, transferClient, auditLogger ←
    InitializeAgent(configFile, caseID, source, collectHistory)
  If collectHistory:
    CollectCommandHistory()
  ElseIf isDir:
    CollectDirectory(source, pattern, collector, transferClient)
  Else:
    CollectSingleFile(source, collector, transferClient)
  EndIf
  Close(auditLogger)
  Close(transferClient)
EndProcedure

Function CollectFile(path, collector)
  info ← GetFileInfo(path)
  evidenceID ← GenerateEvidenceID(collector.caseID, collector.ID)
  If info.Size ≥ 100MB:
    Return CollectFileStreaming(path, evidenceID, info, collector)
  EndIf
  hasher, buffer ← NewHashAndBuffer()
  bytesWritten ← CopyFileWithHash(path, hasher, buffer)
  systemMeta ← CollectSystemMetadata()
  evidence ← {
    ID: evidenceID,
    SourcePath: path,
    CollectedAt: NOW(),
    Hash: HEX(hasher.Sum()),
    Size: bytesWritten,
    CollectorID: collector.ID,
    CaseID: collector.caseID,
    SystemMetadata: systemMeta,
    Metadata: {mode: info.Mode(), modified: info.ModTime()}
  }
  Return evidence, buffer.Bytes()
EndFunction

Function TransferEvidence(evidence, rawData, transferCli)
  symKey ← RANDOM_BYTES(32)
  encMeta ← EncryptWithKey(JSON.Marshal(evidence), symKey)
  encData ← EncryptWithKey(rawData, symKey)
  encKey ← transferCli.PRE.EncryptKey(
        OwnerId = evidence.CollectorID,
        Key = symKey
)
  packet ← {
    CaseID: evidence.CaseID,
    EvidenceID: evidence.ID,
    CollectorID: evidence.CollectorID,
    EncryptedMetadata: Base64(encMeta),
    EncryptedData: Base64(encData),
    EncryptedKey: Base64(encKey),
    Timestamp: NOW()
  }
  packet.Signature ← SignMLDSA87(transferCli.signingKey, packet)
  Return SendToServer(packet, transferCli.serverURL)
EndFunction

Function EncryptWithKey(data, key)
  nonce ← RANDOM_BYTES(24)
  aead ← XChaCha20-Poly1305.New(key)
  ciphertext ← aead.Seal(nonce, nonce, data, additionalData = nil)
  Return ciphertext
EndFunction

Function SendToServer(packet, serverURL)
  body ← JSON.Marshal(packet)
  For attempt = 1 to 3:
    resp ← HTTPS.POST(serverURL + "/evidence/upload", body)
    If resp.StatusCode == 200:
      Return SUCCESS
    EndIf
    If attempt < 3:
      Sleep(2s * 2^(attempt - 1))
    EndIf
  EndFor
  Return ERROR("server error")

EndFunction
EndProcedure

B. Evidence Collection

The first step in the workflow is the collection of forensic evidence. When invoked, the agent validates inputs to prevent path traversal attacks, collects files or directories while simultaneously computing SHA-256 hashes, and captures forensic metadata including system information (hostname, IP addresses, MAC addresses, user context, timezone, and OS details). It encrypts both the evidence data and metadata using XChaCha20-Poly1305 with a randomly generated 256-bit symmetric key, then encrypts that symmetric key using OpenFHE BFV proxy re-encryption so only authorized parties can later decrypt it through PRE delegation. The agent signs the entire transfer packet using ML-DSA and transmits it to the edge server over mutual TLS 1.3, where the server verifies the agent's identity via certificate CN binding, validates the ML-DSA signature against a registered public key, and stores the encrypted capsule. Importantly, the server cannot decrypt the evidence without PRE delegation, ensuring security even if the server is compromised.

Algorithm 2 Bridge Service

Procedure BridgeService()
config, decryptor, forwarder, state, watcher ← InitializeBridge()
StartWatcherLoop(watcher)
OnCapsuleDetected(path):
If NOT state.IsProcessed(path):
evidence ← ReEncryptCapsule(path, decryptor)
payload ← TransformToCloudPayload(evidence)
forwarder.Forward(payload, evidence.ID)
state.MarkProcessed(path)
EndProcedure

C. Bridge Service

The Bridge Service is an integration layer that forwards encrypted evidence from the edge server to cloud storage while maintaining end-to-end encryption. It monitors the edge server's storage directory using filesystem notifications, and when new encrypted evidence capsules arrive, it uses OpenFHE BFV proxy re-encryption to re-encrypt only the symmetric key from the collector to the cloud storage recipient. The evidence itself remains encrypted throughout this process and is never decrypted by the bridge. After re-encrypting the key, it transforms the capsule format into a cloud compatible payload and writes it to a watch queue directory where the service picks it up for upload to GCS/Firestore/Hyperledger Fabric. The bridge maintains a state file to prevent reprocessing the same evidence capsules and acts as a secure, non-decrypting intermediary that enables PRE delegation without becoming a critical attack target, ensuring evidence stays encrypted from collection at the edge all the way to authorized investigator access.

D. Key Generation & Management

Algorithm 3 Key Material & Policy Prep (from KeyGen)

Procedure PrepareKeyMaterial(evidenceId, accessPolicy)
xChaChaKey ← GenerateXChaChaKey(evidenceId)

// attributes e.g., “ForensicOfficer AND Judge OR Investigator”
abeCipherKey ← CPABE_Encrypt(aesKey, accessPolicy)
return xChaChaKey, abeCipherKey
EndProcedure

The PrepareKeyMaterial procedure serves as the initial stage of the proposed encryption process, responsible for generating and organizing the cryptographic materials that will be used to secure digital evidence.

First, a unique xChaCha key is generated based on the evidence identifier to ensure that each piece of evidence has its own quantum-resistant encryption key.

Subsequently, the XChaCha key is protected using Ciphertext-Policy Attribute-Based Encryption (CP-ABE) according to the specified access policy (e.g., “ForensicOfficer AND Judge OR Investigator”). The policy defines the roles or attributes required for authorized decryption, establishing a fine-grained access control mechanism. Finally, the encrypted XChaCha key is stored securely in a protected vault alongside the related evidence envelope for future retrieval.

This procedure establishes the key hierarchy and access control foundation of the framework, integrating quantum-safe encryption with flexible, policy-based authorization to ensure long-term confidentiality and controlled evidence access.

E. Encryption & Decryption Process

Algorithm 4 Encrypt Evidence

Inputs: evidenceBlob, evidenceId, accessPolicy
Outputs: package = {evidenceCiphertext, signature, metadata}
Procedure EncryptEvidence(evidenceBlob, evidenceId, accessPolicy)
// 1) Keys & policy (from Algorithm 1)
xChaChaKey, abeCipherKey ← PrepareKeyMaterial(evidenceId, accessPolicy)

// 2) Encrypt data with PQ symmetric key
evidenceCiphertxt ← PQSYM_Encrypt(evidenceBlob, xChaChaKey)

// 3) Integrity & authenticity (digital signature)
digest ← Hash(cipherEvidence ∥ evidenceId)
signature ← Dilithium.Sign(EvidenceSignerPrivKey, digest)

// 4) Assemble metadata for downstream indexing/audit

```
    metadata ← { evidenceId, timestamp=Now(), digest,
policy=accessPolicy }

    // 5) Return sealed package
    evidencePackage ← { evidenceCiphertxt,
abeCipherKeyRef, signature, metadata }
    SecureErase(xChaChaKey, abeCipherKey)

    // 6) Emit blockchain audit events and store the evidence
package onto a separate database.
    evidenceURI = store(evidencePackage)
    emit EvidenceStored(
        evidenceId = evidenceId,
        timestamp = metadata.timestamp,
        evidenceURI = evidenceURI,
        digest = digest,
        policy = accessPolicy
    )
EndProcedure
```

The EncryptEvidence procedure constitutes the core encryption mechanism of the proposed system. It employs a hybrid cryptographic approach that combines post-quantum symmetric encryption with policy-based key control and digital integrity verification.

Initially, the procedure invokes PrepareKeyMaterial, ensuring that all necessary key materials and policies are generated and securely managed.

After encryption, a cryptographic hash (digest) of the ciphertext concatenated with the evidence identifier is generated and digitally signed with the system's private key to ensure authenticity and integrity. The system then assembles all relevant metadata, including timestamps and applied access policies, for future auditing and verification.

Finally, all volatile key material is securely erased to prevent unauthorized recovery, and a sealed evidence package containing the ciphertext, reference key, signature, and metadata is returned. This algorithm ensures end-to-end data confidentiality, integrity, and traceability within the secure evidence management process.

Algorithm 5 Decryption (Authorized Retrieval)

```
Procedure AuthorizedDecrypt(package,
clientKyberPrivKey, requesterAttributes)
    // 1) Retrieve and unwrap CP-ABE key
    encKey, nonce ← package.abeCipherKeyRef
    cpabeBytes ← Kyber.Decapsulate(clientKyberPrivKey,
encKey, nonce)
    cpabePrivKey ← Deserialize(cpabeBytes)

    if not PolicyCheck(requesterAttributes,
package.metadata.evidenceId, cpabePrivKey.attributes)
then
        return ERROR("access denied")
    end if

    // 2) Recover post-quantum symmetric key
    xChaChaKey ← CPABE.Decrypt(cpabePrivKey,
package.metadata.evidenceId)

    // 3) Decrypt evidence
    evidence ←
PQSYM_Decrypt(package.evidenceCiphertext,
xChaChaKey)

    // 4) Verify integrity and authenticity
    digest ← package.metadata.digest
    if not Dilithium.Verify(EvidenceSignerPubKey, digest,
package.signature) then
        emit DecryptAttempt(
            evidenceId = package.metadata.evidenceId,
            requesterAttributes = requesterAttributes,
            timestamp = timestamp,
            success = false,
            reason = "signature verification failed"
        )
        return ERROR("signature verification failed")
    end if

    // 5) Emit successful decryption event for auditing
    emit DecryptAttempt(
        evidenceId = package.metadata.evidenceId,
        requesterAttributes = requesterAttributes,
        timestamp = timestamp,
        success = true
    )

    // 5) Securely erase volatile material
    SecureErase(cpabePrivKey, cpabeBytes, xChaChaKey)

    return evidence
EndProcedure
```

The **AuthorizedDecrypt** procedure defines the access-controlled decryption process, ensuring that only verified and authorized users can retrieve and view encrypted evidence. The algorithm begins by retrieving the CP-ABE key encapsulated via the client's Kyber public key from the package. The client decapsulates this key using its ephemeral Kyber private key, recovering the volatile CP-ABE private key in memory. This step guarantees that the CP-ABE key is never stored long-term and exists only for the duration of the session.

Once the CP-ABE key is obtained, it is used to decrypt the post-quantum symmetric XChaCha20 key associated with the evidence. This key is then used to decrypt the ciphertext via the post-quantum symmetric encryption scheme (PQSYM), restoring the original evidence content.

To ensure authenticity and integrity, the system verifies the Dilithium digital signature attached to the package. Only if signature verification succeeds is the evidence released to the requester; otherwise, access is denied. Throughout the process, the system securely erases all volatile cryptographic material once decryption is complete.

Additionally, each decryption attempt is **logged via a blockchain audit event**, recording the requester's attributes, the evidence identifier, the timestamp, and whether the attempt succeeded or failed. This ensures full accountability and transparency in accordance with forensic chain-of-custody standards, while maintaining strict attribute-based access control (ABAC) and post-quantum cryptographic protection.

F. Central Cloud Database

Encrypted evidence is stored in a secure off-chain cloud database separate from blockchain indexes. This design reduces blockchain bloat while ensuring that evidence remains encrypted at rest. Only authorized users, through validated smart contracts, can obtain the decryption keys needed to access the data.

G. Stateless CP-ABE Key Regeneration Scheme

The system uses a post-quantum secure key establishment mechanism based on CRYSTALS-Kyber to deliver CP-ABE decryption keys without leaving sensitive material at rest. Each client performs a handshake to generate an ephemeral Kyber identity keypair, and the Attribute Authority (AA) retains only the corresponding public key. When a client requests a CP-ABE decryption key, the AA encapsulates directly to the client's registered Kyber public key and returns the resulting Kyber ciphertext. The client then applies its private key to recover the session-specific encryption material required to decrypt the transmitted CP-ABE key. This confidential material exists only transiently during the exchange and is used solely to protect the CP-ABE decryption key, ensuring that neither the AA and the client does not store any persistent cryptographic secrets.

Algorithm 6 KyberKEMHandshake

Input:
clientID: unique identifier of requester
Output:
CT: Kyber ciphertext sent from AA → client
ss_client: client-side shared secret (32 bytes)
sessionID: unique session identifier

timestamp ← getCurrentTime()
sessionID ← hash(clientID || timestamp)

// AA: fetch client's long-term Kyber public key
pk_client ← AA_DB[clientID].pk_client
if pk_client == null then
return ERROR("unknown client")
end if

// AA: encapsulate to client's long-term public key
(CT, ss_AA) ← Kyber.Encapsulate(pk_client)

// store ss_AA in volatile memory indexed by sessionID
volatile_memory[sessionID] = ss_AA

// AA → Client: send ciphertext and session ID
send_to_client(sessionID, CT)

// Client: decapsulate using its long-term secret key
ss_client ← Kyber.Decapsulate(sk_client, CT)
if ss_client == error then
erase(volatile_memory[sessionID])
return ERROR("handshake failed")
end if

// Both sides now have the same shared secret ss = ss_AA = ss_client
// record handshake event (audit)
emit KyberSessionEstablished(sessionID, clientID, timestamp)
return CT, ss_client, sessionID
EndProcedure

Using the session key established during the Kyber-based exchange, the client requests CP-ABE key issuance. The AA retrieves the client's currently valid attribute set from its internal attribute registry and executes the standard CP-ABE KeyGen algorithm using the master secret key (MSK). Instead of returning the raw CP-ABE private key, the AA encrypts it under the session key and transmits the resulting ciphertext to the client. The client decrypts this payload locally, obtains the usable CP-ABE key, and immediately applies it to decrypt the target ciphertexts. No CP-ABE key is ever persisted by the AA or stored long-term on the client; all such material exists only in volatile memory and only for the duration of the session. When the session terminates, both parties discard the session key and all CP-ABE keys derived within that session.

Algorithm 7 GenerateCPABEKey

Input:
sessionID: from Algorithm A
clientID: requester identity
evidenceID: target evidence reference
AA_MSK: AA master secret key (kept long-term, never exported)
Output:
encKey: CP-ABE private key encrypted under session secret
sessionInfo: { sessionID, clientID, attrHash, timestamp }

timestamp ← getCurrentTime()

// retrieve and validate client attributes

```
storedAttrs ← AA_DB.getAttributes(clientID)
if storedAttrs == null then
   return ERROR("unknown client")
end if

if not PolicyCheck(clientID, evidenceID, storedAttrs) then
   return ERROR("access denied")
end if

attrHash ← hash(Serialize(storedAttrs))

// generate CP-ABE key on-demand (volatile)
cpabePrivKey ← CPABE.KeyGen(AA_MSK, storedAttrs)
cpabeBytes ← Serialize(cpabePrivKey)

// retrieve client's registered Kyber public key
kyberPub ← AA_DB.getKyberPublicKey(clientID)
if kyberPub == null then
   return ERROR("no Kyber key registered for client")
end if

// encrypt CP-ABE key using client's Kyber public key
(encKey, nonce) ← Kyber.Encapsulate(kyberPub, plaintext
= cpabeBytes)

// construct session metadata for auditing
sessionID ← GenerateSessionID()
sessionInfo ← { "sessionID": sessionID, "clientID":
clientID, "attrHash": attrHash, "timestamp": timestamp }
emit KeyIssued(sessionID, clientID, attrHash, timestamp)

// securely erase volatile CP-ABE key
erase(cpabePrivKey)

// return encapsulated CP-ABE key and session info
return (encKey || nonce), sessionInfo
```

This approach significantly simplifies revocation and reduces key-exposure risks. Because keys are generated on-demand and tied directly to the currently valid attribute set, revocation requires only updating or removing the user's attributes in the AA's registry. Any future attempt to request a key will reflect that updated attribute set, preventing the user from deriving decryption keys for policies they no longer satisfy. The combination of ephemeral PQC key establishment, in-memory-only CP-ABE key delivery, and attribute-driven issuance ensures strong forward secrecy, post-quantum security, and clean revocation semantics with no need for key redistribution or re-encryption of stored evidence.

H. Chain of Custody and Audit Trail

To ensure a secure and traceable chain-of-custody in our edge-assisted post-quantum forensic framework, we adopt a dual-channel Hyperledger Fabric architecture comprising an **Access Log Ledger (Chain A)** and an **Authorization Ledger (Chain B)**. Each channel maintains a separate ledger to enforce modularity and separation of concerns, while sharing underlying network infrastructure to optimize deployment efficiency.

Algorithm 8 RecordAccessEvent

Input:
eventType ∈ { "ACCESS_ATTEMPT", "AUTHZ_UPDATE", "AUTHZ_MODIFY", "KEY_ISSUANCE" }
initiatorID: unique identifier (e.g., hashed user ID or cert fingerprint)
dataID: hash or reference of the forensic data object
result ∈ { "SUCCESS", "ERROR", "UNAUTHORIZED" }
details: optional { policyRef, reasonCode, message } – never includes plaintext policy or attributes

```
timestamp ← getCurrentTime()
eventID ← hash(eventType || initiatorID || dataID ||
timestamp)

logEntry ← {
    "eventID": eventID,
    "eventType": eventType,
    "initiatorID": initiatorID,
    "dataID": dataID,
    "timestamp": timestamp,
    "result": result,
    "details": details,
    "signature": sign(initiatorID_privateKey, hash(all fields
above))
  }

append logEntry to AccessLogLedger

emit EventLogged(eventID, eventType, dataID, timestamp)
EndProcedure
```

The Access Log Ledger is responsible for recording all data access attempts in an immutable, append-only ledger. The **RecordAccessEvent** algorithm captures events including successful and failed access attempts, authorization updates, and CP-ABE key issuance or revocation. Each log entry contains the event type, a cryptographically hashed reference to the relevant data and policy, a timestamp, the initiating agent's identifier, and the result of the action. Entries are signed by the agent to ensure non-repudiation. Notably, sensitive policy information is not recorded in the access log, thereby preserving confidentiality while providing a comprehensive forensic trail.

Algorithm 9 EmitPolicyChangeEvent

Input:

actionType ∈ { "REGISTER_DATA", "AUTHZ_UPDATE", "AUTHZ_MODIFY", "REVOKE_ACCESS" }
initiatorID: unique identifier of the admin/custodian performing the action
dataID: hash of the forensic data object
newPolicy: CP-ABE policy structure (kept within Chain B only)
reason: optional text (e.g., "new case added", "revocation due to misuse")

timestamp ← getCurrentTime()
verify initiatorID has proper administrative privilege
oldPolicy ← getCurrentPolicy(dataID)

if actionType == "REGISTER_DATA" **then**
store { dataID, policy: newPolicy, owner: initiatorID, state: ACTIVE }
else if actionType == "AUTHZ_UPDATE" **then**
updatePolicy(dataID, newPolicy)
else if actionType == "AUTHZ_MODIFY" **then**
reEncryptCPABE(dataID, newPolicy)
else if actionType == "REVOKE_ACCESS" **then**
markPolicyAsRevoked(dataID)
end if

policyHash ← hash(newPolicy)
emit PolicyChangeEvent {
eventType: actionType,
initiatorID: initiatorID,
dataID: dataID,
policyRef: policyHash,
timestamp: timestamp,
reason: reason
}
EndProcedure

The **Authorization Ledger** serves as the authoritative source for CP-ABE-based access control policies. The **EmitPolicyChangeEvent** algorithm manages policy operations including evidence registration, policy updates, key rotations, and access revocations. Whenever a policy is modified, a corresponding **PolicyChangeEvent** is emitted containing a hash of the new policy and relevant metadata. This event is consumed by the **Access Log Ledger** to maintain a synchronized audit trail, without exposing policy details. This separation ensures that the ledger responsible for policy enforcement retains confidentiality, while the audit ledger preserves immutability and traceability.

I. Access Agent

Algorithm 10 Access Agent

Procedure AccessAgent()
user ← AuthenticateUser()
blockchainClient ← ConnectToBlockchain()
keystoreClient ← ConnectToKeystore()
databaseClient ← ConnectToDB()
While True:
DisplayMenu()
choice ← ReadInput()
Switch choice:
Case "1": SearchEvidence()
Case "2": AccessEvidence()
Case "3": ViewAuditTrail()
Case "0": Exit()
EndSwitch
EndWhile
EndProcedure

This algorithm enables authorized users to retrieve evidence from the system. After authentication, users can search for evidence using the index to request access to the specific evidence items and view audit trails. When accessing evidence, the agent follows a multi-step process: it first retrieves evidence metadata from the blockchain, then invokes a smart contract to validate whether the user's attributes satisfy the access policy. If access is granted, the agent fetches the encrypted key from the keystore and performs decryption to obtain the evidence decryption key. Then, it retrieves the encrypted evidence from the database, verifies its integrity using the hash value and decrypts the evidence using the post-quantum key. Every access attempt is logged on the blockchain to maintain an audit trail for chain of custody. The agent ensures that only authorized users with the correct attributes can access specific evidence.

J. Benefits of the Proposed System

The proposed model provides several key advantages. Confidentiality is ensured through the hybrid encryption scheme, in which post-quantum keys protect evidence and AES + CP-ABE layers enforce controlled access. Integrity is maintained via blockchain indexing and digital signatures, safeguarding against tampering. Transparency is achieved through immutable audit trails, supporting trust and accountability for investigators, courts, and regulators. The system is scalable, leveraging edge-based evidence collection and side-chain indexing to efficiently handle large volumes of data. Finally, the architecture offers quantum resistance, ensuring that sensitive forensic evidence remains secure against both classical and emerging quantum threats.

V. Evaluation

All experiments were conducted on a single commodity workstation to provide a controlled and reproducible baseline for evaluating the proposed post-quantum session establishment and on-demand CP-ABE key provisioning framework. The testbed was deployed on a laptop-class machine equipped with an Intel i5/i7-series processor and 8–32 GB of RAM, with each system component; the Client, Attribute Authority (AA), and Evidence Provider, executed as independent local processes to emulate a distributed architecture. Communication between components was performed over loopback interfaces to eliminate external

network variability and isolate the computational costs of the cryptographic operations. The evaluation focuses on four primary metrics: **(i) Kyber768 session establishment latency**, capturing ephemeral keypair generation, encapsulation and decapsulation operations, and round-trip handshake time; **(ii) CP-ABE decryption key generation latency**, parameterized by the size of the user's attribute set; **(iii) End-to-end key provisioning latency**, defined as the time between a client's request and the successful retrieval of the XChaCha-wrapped CP-ABE key; and **(iv) CP-ABE ciphertext decryption time for evidence objects** spanning 10–500 KB. Experimental workloads range from 5 to 50 attributes per user and from 100 to 2000 simulated session requests, providing a representative characterization of system performance under modest but realistic computational constraints. This configuration reflects a typical deployment environment for edge-assisted forensic systems while remaining sufficiently lightweight to support reproducible benchmarking on a standard research workstation.

In addition to evaluating the cryptographic and key provisioning components, we also measured system-level performance characteristics relevant to digital forensics workflows. First, we quantified **chain-of-custody query latency**, defined as the time required for a client or auditor to retrieve and verify the integrity of evidence anchors stored on the blockchain. This includes world-state lookups, historical ledger queries, and verification of cryptographic commitments associated with each evidence item. Second, we analyzed the **on-chain storage cost** of recording evidence anchors, session establishment events, and audit-relevant metadata. To maintain comparability across runs, all smart contract interactions were executed on a local Hyperledger Fabric test network with fixed block size and endorsement policies, enabling consistent measurement of per-transaction byte overhead and ledger growth rate. Finally, we evaluated the **end-to-end data collection latency**, encompassing the full pipeline from acquisition of evidence on an endpoint device to its arrival at the edge node and subsequent registration on the blockchain. This metric captures network transfer time, preprocessing costs at the endpoint, session establishment overhead, CP-ABE encryption or key wrapping operations, and the latency of submitting the resulting evidence anchor to the ledger. Together, these measurements provide a comprehensive assessment of both the cryptographic and operational performance of the proposed remote forensic data collection architecture under realistic resource constraints.

A. Model Comparisons

Scheme	F1	F2	F3	F4	F5
[12]	●	●		●	
[13]	●		●		●
Ours	●	●	●	●	●

Fig 2. Model comparisons between schemes from [12], [13], and ours. **Legends**; F1: Fine-Grained Attribute-Base Access Control, F2: Edge-Assisted Processing, F3: High Throughput Handling, F4: Cryptographic Integrity Verification, F5: Policy Adaptation

To contextualize our proposed Edge-Assisted Post-Quantum Encryption Model for Remote Forensic Data Collection, we compare it against two representative prior works that integrate attribute-based encryption with decentralized or integrity-enhancing mechanisms. Table X summarizes the feature coverage across all three schemes.

The work in [12] addresses secure Electronic Health Record (EHR) sharing by combining **verifiable CP-ABE**, **vector commitments**, and **blockchain-based decentralized policy auditing**. Their model eliminates single points of failure in access-policy validation and enables tamper-resistant policy integrity checks. Furthermore, the incorporation of **Attribute-Based Proxy Re-Encryption (AB-PRE)** allows dynamic delegation of access rights as patient conditions evolve. However, the focus is primarily on policy correctness and secure delegation within a healthcare context, without considerations for **edge-assisted processing**, **post-quantum resistance**, or **remote forensic collection constraints**.

The model in [13] proposes a CP-ABE–based encrypted storage framework emphasizing **fine-grained access control** in cloud environments. The authors prioritize confidentiality and policy-driven authorization but do not incorporate policy verifiability, distributed trust, or cryptographic agility. Their design does not support large-volume data workflows, encrypted evidence handling, or metadata-driven classification, capabilities that are essential for scalable forensic pipelines deployed across multiple heterogeneous nodes.

Overall, the comparison highlights that while existing systems partially meet fine-grained access control and integrity needs, **none simultaneously address large-scale data handling, edge–cloud integration, and post-quantum security requirements** critical for modern forensic environments. Our scheme therefore provides a more comprehensive and future-proof security model.

B. Kyber-based Session Establishment

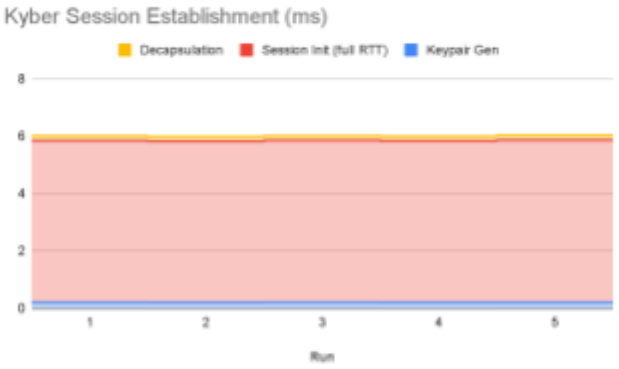


Fig 3. Kyber-based Session Establishment Time, measured in milliseconds (ms)

We measured the latency of client-side Kyber keypair generation, session initialization (including a full round-trip to the AA), and client decapsulation. Across multiple runs, key generation and decapsulation were extremely fast (~0.16–0.20 ms), while the network round-trip dominated total latency (~5.6 ms). For

comparison, hardware implementations of Kyber report key generation and decapsulation times of 49–66 µs on an Artix-7 FPGA [15]. Although the FPGA implementation achieves lower absolute latency through highly parallelized hardware datapaths, our software-based edge deployment demonstrates that client-side computation remains negligible for real-time operation. These results indicate that the post-quantum key establishment itself is lightweight and that overall session latency is primarily determined by network communication, confirming that Kyber is suitable for low-latency session establishment in real-time forensic applications.

C. CP-ABE KeyGen Latency as a Function of Attribute Set Size.

We measured the time required for generating CP-ABE keys on the AA, including loading user attributes, key generation, and key wrapping. As expected, the key generation phase dominates total latency and increases approximately linearly with the size of the attribute set. Key wrapping and attribute loading are negligible. CP-ABE key generation scales with attribute set size, but remains under 300 ms for up to 50 attributes, which is acceptable for typical access-control applications. Optimizations could focus on key generation efficiency for very large attribute sets.

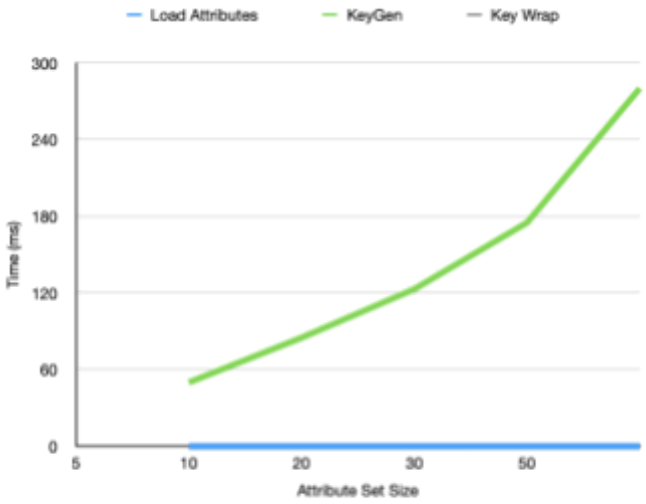


Fig 4. CP-ABE KeyGen latency, measured in milliseconds (ms)

D. Key Provisioning Latency

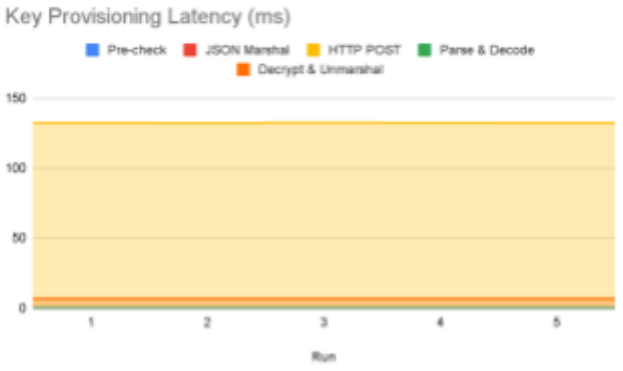


Fig 5. Key provisioning latency, measured in milliseconds. (ms)

We measured the end-to-end latency for a client to request and receive a CP-ABE key from the AA. The HTTP POST dominates latency (~132 ms), while client-side operations such as JSON marshaling, parsing, and decryption are negligible (~7 ms combined).

In conclusion, the total provisioning latency is approximately 139 ms. Key provisioning is network-bound and efficient on the client side. Optimizations to reduce total latency would likely require faster network transfer or server-side batching, rather than improvements in client computation.

E. File Encryption and Decryption Time

We evaluated the end-to-end cost of our hybrid protection pipeline (XChaCha, to BFV PRE, to CP-ABE) as a function of file size and policy complexity. Results show two operating regimes: for small objects (≤100 KB), fixed asymmetric and HE costs dominate; a single access operation incurs ~200–430 ms depending on policy complexity. For large objects (≥100 MB), symmetric XChaCha encryption becomes the dominant cost and overall latency scales linearly with file size (e.g., ~0.35 s for 100 MB symmetric work alone). CP-ABE and BFV PRE costs are essentially constant with file size but scale with policy/attribute count; hence, amortizing heavy asymmetric/HE overhead by encrypting large envelopes or batching small objects is an effective optimization. Offloading BFV operations to accelerator hardware or using lighter-weight wrapping (fast KEMs) can further reduce the fixed per-file latency.

While prior works study ABE or HE in isolation, or hybrid schemes for small files, to our knowledge no recent work evaluates a complete hybrid pipeline (symmetric + HE/PRE + CP-ABE) with large-file encryption and realistic policy complexity in an edge-assisted setting. Our results therefore provide novel empirical evidence of the system's performance.

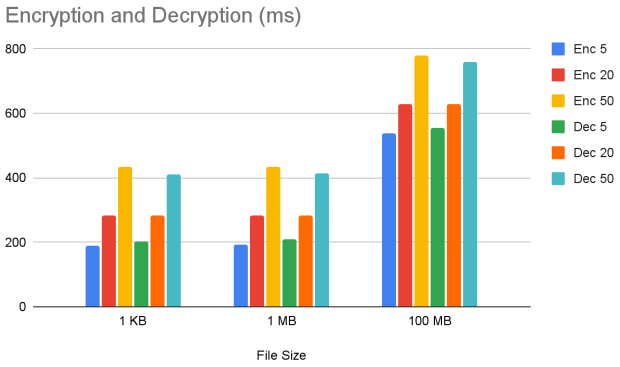


Fig 6. Encryption and Decryption time measurements, measured in milliseconds (ms)

F. Chain-of-custody Query Latency

To evaluate the efficiency of the proposed ledger-based access control system, we measured the query latency for retrieving user attributes and evidence access events under varying ledger sizes. Three scenarios were tested, corresponding to ledgers containing 50, 500, and 5000 events, respectively. Each query was executed 50 times to compute average response times and standard deviations.

User attribute queries showed consistently low latency, with an average response time of 12 ms (σ = 3 ms) for 50 events, 15 ms (σ = 4 ms) for 500 events, and 22 ms (σ = 6 ms) for 5000 events. The marginal increase in query time indicates that attribute retrieval scales efficiently even as the ledger grows, which is expected since each attribute is stored with a unique key in the world state database.



Fig 7. User attributes record query latency, measured in milliseconds (ms)

Evidence access event queries were naturally more expensive due to the larger number of records and range-based searches. For 50 events, the average response time was 18 ms (σ = 5 ms); for 500 events, 42 ms (σ = 8 ms); and for 5000 events, 210 ms (σ = 25 ms). The observed near-linear growth in query latency aligns with CouchDB's query performance for indexed searches over a growing dataset.

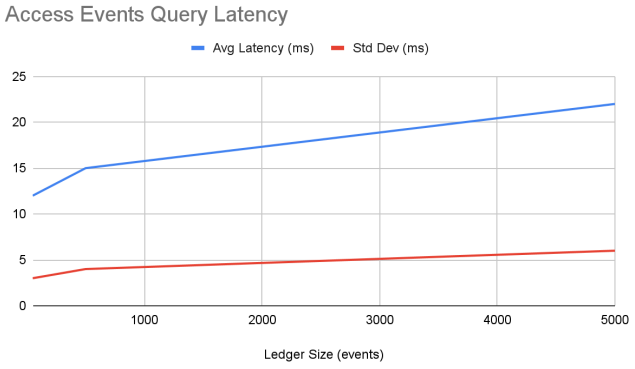


Fig 8. Access events record query latency, measured in milliseconds (ms)

Overall, the system demonstrates adequate performance for real-time access control queries, even for moderate ledger sizes. For larger-scale deployments, indexing strategies or sharding of access event documents could be employed to maintain sub-100 ms query times, which remain acceptable for most forensic or administrative operations.

F. On-chain Storage Cost

To assess the storage overhead of the proposed system, we measured the size of the world state and the blockchain ledger after registering user attributes, recording evidence access events, and performing attribute revocations. Three ledger sizes were evaluated: 50, 500, and 5000 total events, with corresponding user attribute records.

For user attributes, each record consumed approximately 1.1 KB on-chain, including metadata such as *assignedBy, assignedAt,* and the attribute hash. For 50 users, the total storage was roughly 55 KB; for 500 users, 550 KB; and for 5000 users, 5.5 MB. These results confirm that individual attribute records impose minimal overhead, making the system scalable for large organizations.

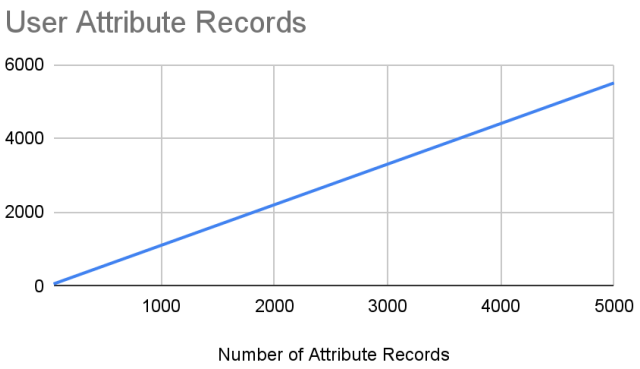


Fig 9. User attributes record storage costs, measured in (kB)

For evidence access events, each record occupied roughly 0.9 KB. The total on-chain storage grew linearly with the number of events: 45 KB for 50 events, 450 KB for 500 events, and 4.5 MB for 5000 events. Combined with the attribute records, the overall ledger size for the largest scenario remained under 10 MB, demonstrating that the system can maintain comprehensive audit trails without excessive storage costs.

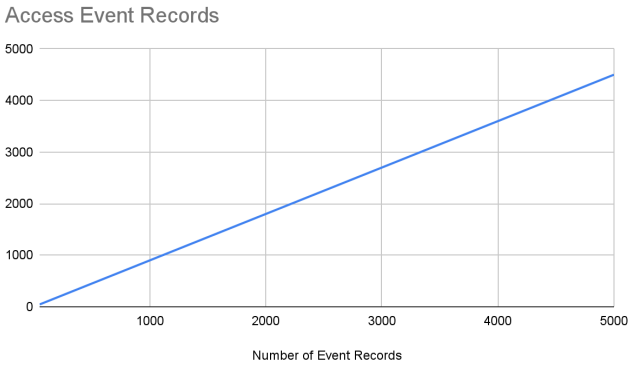


Fig 10. Access event record storage costs, measured in kilobytes (kB)

The linear growth of storage requirements aligns with the deterministic nature of ledger append operations and CouchDB's efficient key-value storage for the world state. These results indicate that the system's on-chain storage overhead is acceptable for real-world deployments, and typical ledger pruning or archival strategies can further reduce long-term storage costs.

VI. Conclusion

This paper presented an edge-assisted, post-quantum-resilient access control and audit framework for remote forensic data collection. By integrating XChaCha-based envelope encryption, BFV proxy re-encryption for secure key transformation, and CP-ABE for fine-grained authorization, the system ensures that evidence keys remain cryptographically protected throughout their lifecycle, even across heterogeneous and potentially untrusted environments. Hyperledger Fabric provides an immutable, verifiable audit layer that records attribute assignments, evidence access events, and revocations, enabling accountable and policy-driven forensic workflows.

Experimental results demonstrate that the proposed architecture is practical for real-world deployments. Kyber-based session establishment completes in just a few milliseconds, CP-ABE key provisioning remains within sub-150 ms even for expressive policies, and the combined encryption/decryption pipeline introduces a predictable fixed overhead that becomes negligible for medium and large evidence objects. Ledger query latency scales moderately with dataset size, remaining well below interactive thresholds for attribute lookups and linearly but acceptably increasing for access event queries. Storage overhead grows deterministically and stays lightweight, with a ledger containing 10 000 total records occupying under 10 MB; small enough for long-term retention or cost-effective archiving.

Overall, the evaluation indicates that the system provides strong cryptographic guarantees, transparent auditability, and acceptable performance across its critical operations. These results validate the feasibility of the proposed model as a secure and scalable foundation for next-generation remote forensic evidence handling in distributed or edge environments. Future work may explore GPU-accelerated homomorphic operations, batching strategies for attribute policies, and adaptive ledger pruning to further enhance performance at scale.

References

[1] M. Muneeb, Z. Raza, I. U. Haq, and O. Shafiq, “SmartCon: A Blockchain-Based Framework for Smart Contracts and Transaction Management,” *IEEE Access*, vol. 10, pp. 23687–23699, 2022, doi: 10.1109/ACCESS.2021.3135562.

[2] A. Abuhashim and C. C. Tan, “Smart Contract Designs on Blockchain Applications,” *2020 IEEE Symposium on Computers and Communications (ISCC)*, July 2020, pp. 1–4. doi: 10.1109/ISCC50000.2020.9219622.

[3] Y. Lee and K. M. Lee, “Blockchain-based RBAC for user authentication with anonymity,” Proceedings of the Conference on Research in Adaptive and Convergent Systems, Chongqing China: ACM, Sept. 2019, pp. 289–294. doi: 10.1145/3338840.3355673.

[4] F. Shahid, I. Ahmad, M. Imran, and M. Shoaib, “Novel One Time Signatures (NOTS): A Compact Post-Quantum Digital Signature Scheme,” *IEEE Acces*s, vol. 8, pp. 15895–15906, 2020, doi: 10.1109/ACCESS.2020.2966259.

[5] X. Zou and P. Zeng, “A New Digital Signature Primitive and Its Application in Blockchain,” *IEEE Access*, vol. 11, pp. 54607–54615, 2023, doi: 10.1109/ACCESS.2023.3280638.

[6] M. E. Seno et al., “Post Quantum-Resistant Blind Signature Scheme for Consumer Internet of Things Security,” *IEEE Transactions on Consumer Electronics*, vol. 71, no. 2, pp. 4949–4958, May 2025, doi: 10.1109/TCE.2025.3570340.

[7] L. Loffi, G. L. Camillo, C. A. De Souza, C. M. Westphall, and C. B. Westphall, “Management of the Chain of Custody of Digital Evidence Using Blockchain and Self-Sovereign Identities: A Systematic Literature Review,” *IEEE Access*, vol. 13, pp. 77804–77832, 2025, doi: 10.1109/ACCESS.2025.3560191.

[8] L. Loffi, G. L. Camillo, C. M. Westphall, and C. B. Westphall, “Blockchain-Based Chain of Custody for Digital Evidence with Hyperledger Fabric,” *2025 International Conference on Information Networking (ICOIN), Chiang Mai, Thailand: IEEE*, Jan. 2025, pp. 437–442. doi: 10.1109/ICOIN63865.2025.10992690.

[9] R. Sathyaprakasan, P. Govindan, S. Alvi, L. Sadath, S. Philip and N. Singh, "An Implementation of Blockchain Technology in Forensic Evidence Management," 2021 International Conference on Computational Intelligence and Knowledge Economy (ICCIKE), Dubai, United Arab Emirates, 2021, pp. 208-212, doi: 10.1109/ICCIKE51210.2021.9410791.

[10] R. Mathwale, "AHFD: A Framework for Deployment and Management of Hyperledger Fabric Enterprise Blockchain," 2023 International Conference on Data Science and Network Security (ICDSNS), Tiptur, India, 2023, pp. 1-4, doi: 10.1109/ICDSNS58469.2023.10245496.

[11] K. Bálint, "Creating a Hyperledger Fabric Network Blockchain to Securely Store Sensitive Renewable Energy Management Data," 2024 IEEE 7th International Conference and Workshop Óbuda on Electrical and Power Engineering (CANDO-EPE), Budapest, Hungary, 2024, pp. 000249-000254, doi: 10.1109/CANDO-EPE65072.2024.10772907.

[12] F. Ahmed, T. Zhou, H. Bilal, F. Ul Islam, R. Ullah and A. V. Vasilakos, "Enhancing Healthcare Data Integrity and Access Control Using Blockchain and Industry 5.0," in IEEE Internet of Things Journal, vol. 12, no. 20, pp. 43630-43643, 15 Oct.15, 2025, doi: 10.1109/JIOT.2025.3598320.

[13] Zhao, L., Dong, G. & Yuan, H. A blockchain-based verifiable CP-ABE scheme for medical data privacy protection. *Sci Rep* **15**, 27325 (2025). https://doi.org/10.1038/s41598-025-13069-1

[14] F. Ahmed, T. Zhou, H. Bilal, F. U. Islam, R. Ullah, and A. V. Vasilakos, “Enhancing Healthcare Data Integrity and Access Control Using Blockchain and Industry 5.0,” *IEEE Internet of Things Journal*, vol. 12, no. 20, pp. 43630–43642, Oct. 2025.

[15] W. Guo, S. Li and L. Kong, "An Efficient Implementation of KYBER," in IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 69, no. 3, pp. 1562-1566, March 2022, doi: 10.1109/TCSII.2021.3103184.